Trilateration

TVR Rocket

April 2026

1 Introduction

Given a set of at least 4 tags with positions $(n_{i,x}, n_{i,x}, n_{i,x})$, which are at a distance $n_{i,d}$ from our rocket, we want to solve for the coordinates of our rocket. We will proceed first assuming that we are given exactly four tags with exactly correct values, and will extend our method to allow for arbitrary tags with slight errors afterwards.

1.1 $n = 4$

$$
\begin{aligned}
(x - n_{1,x})^2 + (y - n_{1,y})^2 + (z - n_{1,z})^2 &= n_{1,d}^2 \\
(x - n_{2,x})^2 + (y - n_{2,y})^2 + (z - n_{2,z})^2 &= n_{2,d}^2 \\
(x - n_{3,x})^2 + (y - n_{3,y})^2 + (z - n_{3,z})^2 &= n_{3,d}^2 \\
(x - n_{4,x})^2 + (y - n_{4,y})^2 + (z - n_{4,z})^2 &= n_{4,d}^2
\end{aligned}
$$

Expanding the first two, we get:

$$
\begin{aligned}
x^2 - 2xn_{1,x} + n_{1,x}^2 + y^2 - 2yn_{1,y} + n_{1,y}^2 + z^2 - 2zn_{1,z} + n_{1,z}^2 &= n_{1,d}^2 \\
x^2 - 2xn_{2,x} + n_{2,x}^2 + y^2 - 2yn_{2,y} + n_{2,y}^2 + z^2 - 2zn_{2,z} + n_{2,z}^2 &= n_{2,d}^2
\end{aligned}
$$

Subtracting the second from the first:

$$
\begin{aligned}
&2x(n_{2,x} - n_{1,x}) - n_{2,x}^2 + n_{1,x}^2 \\
&+2y(n_{2,y} - n_{1,y}) - n_{2,y}^2 + n_{1,y}^2 \\
&+2z(n_{2,z} - n_{1,z}) - n_{2,z}^2 + n_{1,z}^2 \\
&= n_{1,d}^2 - n_{2,d}^2
\end{aligned}
$$

We can repeat this, subtracting the ith from the first to get a system of equations that we can plug into a matrix.

1.2 $n > 4$

That kind of blows, though. To allow for an arbitrary number of points, and to ensure the algorithm works even given that our distances will include a small error term, we can turn this into an optimization problem. An overview can be found here. Let p_i be a $\mathbb{R}^3 \to \mathbb{R}$ function that takes a prediction of our current coordinate and outputs the distance to tag i. For instance, for $i = 1$:

$$(p_1(x, y, z))^2 = (x - n_{1,x})^2 + (y - n_{1,y})^2 + (z - n_{1,z})^2$$

We can then construct an overall error function by averaging the squares of the difference between our prediction and the actual distance for all tags i:

$$e(x, y, z) = \frac{1}{n} \sum_{i=1}^{n} (p_i(x, y, z) - n_{i,d})^2$$

Great, now we just need to minimize this error function. We can do this using standard techniques from numerical analysis. The basic idea is that we start with some prediction (we will always have a decent prediction; the last known position of the rocket, potentially with accel data added on, or the origin if we're just starting the rocket).

Given that this is a function of only three variables, we will just use Newton's method (for now). This only requires us to invert a 3x3 matrix, which we can actually precompute. There exist methods that are more computationally efficient, but our dimension is quite small so we'll just ignore that for now and let codex deal with it.

1.3 Newton's Method

We want to find the point (x_0, y_0, z_0) that minimizes $e(x, y, z)$. We will have some initial guess, (x_g, y_g, z_g). We want to find some adjustment vector Δv that brings us to the local minima. Just like in single variable calculus, we can find where the linear approximation of our function is 0, corresponding to a minima.

$$e(x_0, y_0, z_0 + \Delta v) \approx e(x_0, y_0, z_0) + J(\Delta v)$$

Where J is the Jacobian of e. We want this linear approximation to be equal to 0, allowing us to solve for Δv:

$$\begin{aligned} e(x_0, y_0, z_0) + J(\Delta v) &= 0 \\ \implies J(\Delta v) &= e(x_0, y_0, z_0) \\ \implies \Delta v &= (J)^{-1} e(x_0, y_0, z_0) \end{aligned}$$

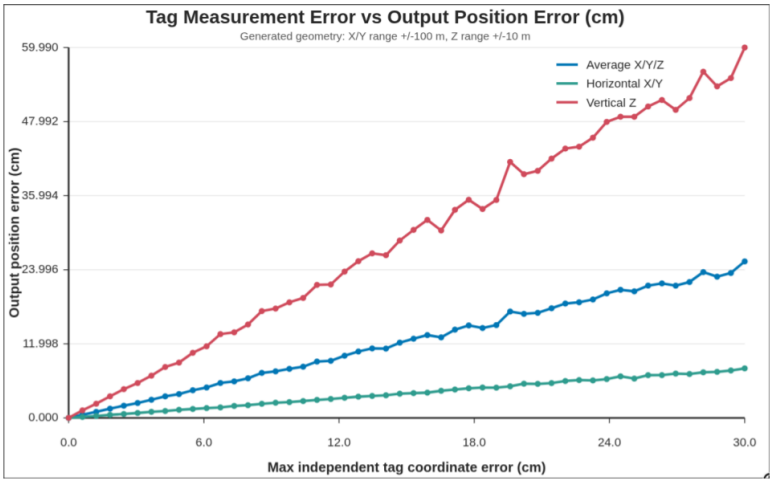
Then our new guess for v is $v + \Delta v$. Given that the linear approximation is only an approximation, this isn't going to be an exact solution so we'll have to repeat this process a couple of times.

This jacobian is VERY annoying, so I will not write it all out here.

After you have these basics, there are a lot of fancy optimizations you can apply (thank you codex) to improve runtime speed & accuracy.

1.4 Practical Considerations

This process assumes that we know the exact position of each tag. We can get kind of close by physically measuring them, but will it be close enough? By default, these are our error levels:



At about 10 cm of error (per axis), we get about 6-8cm of error in the reported position (per axis). This seems acceptable, so no finicky stuff is required yet. We'll just have to bring a meter stick and protractor.

If the position of 4 tags is known (perhaps 4 tags close to the drone), we can use a reverse trilateration process to determine the position of the other tags, but the accuracy of this process remains to be seen (possibly worse accuracy; would only save time).